

Machine Learning and Agentic AI for Macroeconomic Research

9: Agentic AI and Claude Code

Zhigang Feng

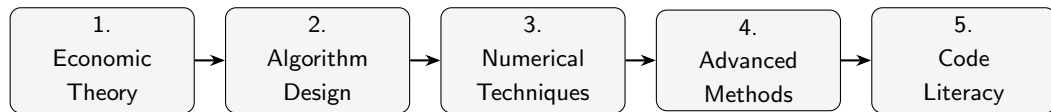
Spring 2026

Lecture Agenda

1. What is agentic AI? (vs. LLM, vs. RAG)
2. Claude Code: under the hood
3. The 5-pillar workflow (detail)
4. Tool landscape + Codex comparison
5. The homotopy workflow
6. The iterative workflow
7. Project structure: `.claude/`
8. Beginner mistakes to avoid
9. Skills, agents, sub-agents
10. Tracing Aiyagari end-to-end
11. Case studies + your first exercise
12. How research changes + limitations
13. Wrap-up

The throughline: the *5-pillar workflow*—introduced on the next slide—is referenced on every section divider in this lecture.

The 5-Pillar Workflow



You provide: formulation, design, validation

AI provides: syntactically correct code, debugging, refactoring

This is the throughline. Every section in this lecture comes back to one of these five pillars. Mastering them is what makes you the *research architect*—AI handles the bricks; you design the structure.

What is Agentic AI?

The Naive Baseline: ChatGPT in a Browser Tab

You ask in a chat window:

“Pull the FOMC meeting dates since 2008 and plot the federal-funds-rate path around each meeting.”

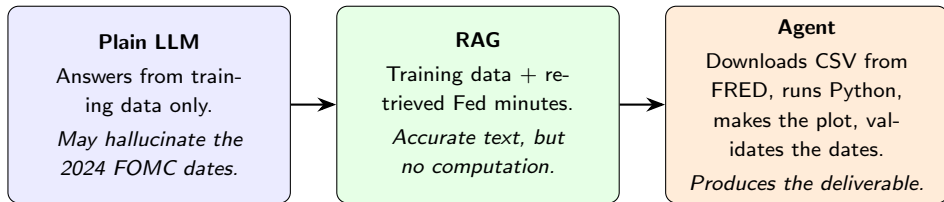
A plain chatbot will write Python that *looks* reasonable. But it cannot:

1. **Execute** the code—it has no Python interpreter
2. **Fetch** the data—it has no network or file access
3. **Iterate** on real output—it never sees the error messages or the wrong dates
4. **Persist** state—tomorrow it forgets your project conventions and variable names

The result: you become the human shuttle—copy code into your editor, run it, copy the error back, paste the new code, repeat. *This is what we are about to fix.*

Chatbot → RAG → Agent: a Capability Spectrum

One question across all three: *"Plot the federal-funds-rate path around recent FOMC meetings."*



Each step adds capability without removing anything. An agent *contains* an LLM and can call retrieval; the spectrum is cumulative. You will use all three in a single research project.

What Makes Something “Agentic”?

Four defining features—each illustrated with an empirical-economics example:

- **Tool use:** invokes `pandas`, `curl`, `R`, `statsmodels`, the shell—whatever the task requires
- **Autonomy:** plans a multi-step empirical workflow without re-prompting
 - *e.g.*, “download FRED → clean → regress → plot → save table” as one command
- **Iteration:** observes that a regression failed, identifies the bad column, fixes the formula, reruns
- **Persistence:** remembers your project conventions across sessions (variable naming, preferred packages, validated benchmarks)

All four features are cumulative. Tool use without autonomy = a glorified Python REPL.

Autonomy without iteration = a script that runs once and dies on the first error. You need all four for “agentic.”

The Bridge: How a Chatbox Becomes a Terminal Agent

One key technical change: modern LLMs (post-2023) support **structured tool calls**. Instead of writing “you should run `python solve.py`,” the model emits a machine-readable request:

```
{ "tool": "Bash", "command": "python aiyagari_v0.py" }
```

A **harness**—software installed on your Mac—executes this and returns the real output. The model reads it and decides what to do next. This loop is what makes something “agentic.”

	claude.ai (browser)	Claude Code (terminal)
LLM model	same (Opus 4.6)	
Harness	none (sandboxed)	local CLI on your Mac
Can run Python?	no	yes— <i>your</i> Python, <i>your</i> machine
Sees real errors?	no	yes—loops back automatically

Same brain, different arms. The terminal agent is not a smarter model—it is the *same* model given real tools. The harness is what flips the switch from text-only to text + actions.

Standalone LLM vs. RAG vs. Agent

	Plain LLM	LLM + RAG	Agent
Knowledge source	Training data	Training + retrieved docs	Training + retrieved docs + live data fetched at runtime
Action capability	None (text only)	None (text only)	Reads/writes files, runs code, hits APIs
State persistence	Per-conversation	Per-conversation	Cross-session via files
Latency	Seconds	Seconds	Minutes (multi-step)
Typical economic use	Brainstorming model setup	Literature review of Fed minutes	Full empirical pipeline (FRED → figure)

Different tools for different stages. You will use all three in a single research project: an LLM to design the model, RAG to find relevant papers, an agent to run the regression and make the figure.

The Ladder of AI Coding Maturity

Level 5: long-running unattended agents (60+ minutes on a task)

Level 4: Model Context Protocols (MCPs)—tool integrations (databases, APIs)

Level 3: *terminal coding agents (Claude Code, Aider, Gemini CLI)*—this lecture

Level 2: IDE-based agents (Cursor, GitHub Copilot)—Tab completion, file-aware chat

Level 0–1: copy/paste between ChatGPT (browser) and your editor

Where most economists are today: Level 1–2.

Where this lecture takes you: Level 3.

Adapted from Paul Goldsmith-Pinkham, “Getting Started with Claude Code.”

Why Level 3 is the Sweet Spot for Empirical Research

At Level 3 you get **tool use + autonomy + iteration**, but you still stay in the loop to validate.

- **Below Level 3** (browser chat / Cursor)

Too manual: you re-explain the project every session; the AI never sees its own output

- **Level 3** (Claude Code)

Right balance: the AI runs the code, sees real errors, fixes them—but a human approves each major step

- **Above Level 3** (long-running unattended agents)

Too risky for research code: nobody is watching when the agent silently changes a regression specification or grid choice

Concrete example: a single Aiyagari $V0 \rightarrow V1 \rightarrow V2$ cycle—design $\rightarrow$ implement $\rightarrow$ validate $\rightarrow$ extend—is exactly the kind of bounded, validated task that Level 3 was built for.

Claude Code in Context: Alternatives

Claude Code is one of several Level-3 terminal agents. Each has a similar architecture:

Tool	One-line description
Claude Code	Anthropic's terminal agent. Largest context window. Strong reasoning. Used for the rest of this lecture
Gemini CLI	Google's terminal agent. Free tier with Gemini Pro. Tight Google-services integration
OpenAI Codex CLI	OpenAI's terminal agent. Tight ChatGPT-Plus integration
Aider	Open-source terminal agent. Bring-your-own model (Claude / GPT / local). Good git integration
Cursor (Level 2)	Most popular IDE-integrated agent. Tab completion + chat + multi-file composer

The workflow we teach transfers to any Level-3 agent. We use Claude Code as the concrete example because the file-based harness (CLAUDE.md, .claude/) is the best documented and most stable.

Codex vs. Claude Code: A Budget-Aware Comparison

Panjwani's (2026) recommendation for economists starting today on a tight budget: **start with Codex.**

	OpenAI Codex	Claude Code
Model (2026)	GPT 5.4	Claude Opus 4.6
\$20/mo tier	Higher usable throughput	More restrictive rate limits
Desktop app	Polished GUI for most users	Terminal-native (power users)
Context window	Shorter	Largest available (~200K tokens)
Harness files	AGENTS.md, .agents/skills/	CLAUDE.md, .claude/skills/

Important caveat: *"This space changes fast—today it's Codex, tomorrow it's back to Claude, the day after something else."* The recommendation is for *this moment*, not permanent truth.

Source: Panjwani (2026). He has switched back and forth between the two tools over time.

Skills Are an Open Standard

You are **not** locked into an ecosystem. The workflow concepts transfer cleanly:

Concept	Claude Code	Codex
Project context file	CLAUDE.md	AGENTS.md
Skills directory	.claude/skills/	.agents/skills/
Rules directory	.claude/rules/	.agents/rules/
Agent definitions	.claude/agents/	.agents/agents/

What this means in practice:

- A colleague's skill written for Codex works in Claude Code with minimal adaptation
- If you switch tools next month, your project harness migrates with you
- The *workflow*—plan first, encode skills, validate against benchmarks—is tool-agnostic

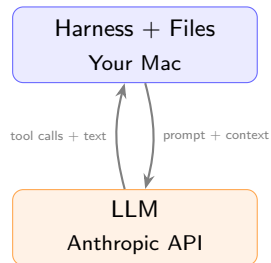
This reinforces the main claim: the workflow we teach in this lecture transfers to *any* Level-3 agent. Learn the pattern, not the brand.

Claude Code: Under the Hood

Three Components of Claude Code

Claude Code has three physically distinct parts:

1. **The Harness** (lives on your Mac)
CLI software you install. Reads your files, runs shell commands, manages the conversation loop. This is what “Claude Code” means when installed.
2. **The LLM** (lives on Anthropic’s servers)
Every message is bundled with conversation history and sent via HTTPS to the API. The “thinking” happens in the cloud.
3. **Your Files** (live on your Mac)
Python scripts, data CSVs, LaTeX—all on your disk. Code executes on your hardware with your packages.



Key: code runs locally.
Only the *conversation* traverses the network.
Your data never uploads.

The Mac File Hierarchy: Global vs. Local

Two layers of configuration. **Local overrides global.**

Path	Scope	What lives here
<code>~/.claude/settings.json</code>	Global	Default permissions, model preference
<code>~/.claude/MEMORY.md</code>	Global	Cross-project memory (persists forever)
<code>~/.claude/skills/</code>	Global	Skills available in every project
<code><project>/CLAUDE.md</code>	Local	Project brain, read every session
<code><project>/.claude/settings.json</code>	Local	Project permissions (overrides global)
<code><project>/.claude/skills/</code>	Local	This project's slash commands
<code><project>/.claude/agents/</code>	Local	Specialized reviewer roles
<code><project>/.claude/rules/</code>	Local	Always-on guardrails

On a Mac: `~` = `/Users/yourname`. To see what you have: `ls ~/.claude/`

Practical rule: global `~/.claude/` is your personal defaults (apply everywhere). Local `.claude/` is the project's rules (apply only here). Set permissions you always want globally; set project-specific conventions locally.

Initialization: What Happens When You Type `claude`

Five steps run **before you type your first message**:

1. **Read global config**

Loads `~/.claude/settings.json`: permission defaults, model preference

2. **Detect project context**

Searches current directory (and parents) for `CLAUDE.md` and `.claude/`

3. **Load project brain**

Reads `CLAUDE.md` into the context window—Claude now “knows” the project

4. **Auto-load all rules**

Every `.claude/rules/*.md` file is loaded silently—no action required from you

5. **Ready**

Context window is pre-populated with project knowledge before you say anything

Analogy: step 4 is a professor re-reading lecture notes before class. Students arrive; context is already loaded.

Cost warning: steps 2, 4 consume tokens before you type anything. Keep `CLAUDE.md` and rules files

The Permission System and Local Code Execution

Claude's toolbox:

Tool	What it does
Bash	Run any shell command
Read	Read a file from disk
Write	Create or overwrite a file
Grep/Glob	Search content / find files
Agent	Spawn a child Claude instance

The permission model:

- **Default:** Claude asks before every Bash call. You see the exact command before it runs.
- **Whitelist:** add trusted commands to `.claude/settings.json`:

```
"allowedTools": [  
  "Bash(python *)",  
  "Bash(git *)" ]
```

- **Bypass-permissions:** skips all prompts. Use only when you fully trust the task.

“Which Python ran this?” (common confusion):

- Claude's Bash inherits your shell's \$PATH
- If you ran `conda activate myenv` before claude, *that* Python runs—not a special Anthropic Python
- To confirm: add a `which python` check to your CLAUDE.md conventions

Key Modes: Plan, Effort, and Session Control

Command / Key	What it does	When to use
Shift+Tab	Plan mode: Claude writes a plan, does NOT execute. Waits for your approval.	Start every non-trivial task here
/effort max	Maximum reasoning depth: highest quality, slower, costs more	Novel problems, economic correctness
/effort default	Balanced quality and speed	Routine edits, simple queries
/cost	Show tokens used + estimated USD for this session	Budget check before long tasks
Esc	Interrupt Claude mid-task immediately	When it's going the wrong way
/compact	Summarize conversation to free context window	After ~20 turns; between phases
/clear	Wipe context, start fresh	New unrelated task

The effort–cost tradeoff:

- /effort max + long context $\approx$ \$5–15 per complex session (Opus 4.6 pricing)
- Spend effort on *decisions*: algorithm choice, debugging wrong economics, code review
- Save effort on *execution*: renaming variables, reading files, small edits

The 5-Pillar Workflow (Detail)

Pillar 1: Economic Theory

You must **formulate the economic problem precisely**:

Household optimization:

- Write the objective: $\max \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t u(c_t, n_t)$
- Specify constraints: budget, borrowing limits, information structure
- Derive optimality conditions: Euler equations, transversality

Equilibrium concepts:

- Define competitive equilibrium: optimality + market clearing
- Recursive competitive equilibrium for computation
- Partial vs. general equilibrium

Aiyagari anchor: *Without a precisely written household problem (CRRA utility, asset state, borrowing constraint), the AI's Aiyagari code may solve a subtly different model—and you will only catch it if you wrote down the equations first.*

Pillar 2: Algorithm Design

You must know **how to translate theory into computation**:

Dynamic programming approach:

- Bellman equation: $V(s) = \max_a \{u(s, a) + \beta \mathbb{E}[V(s')]\}$
- Value function iteration (VFI): iterate $T(V) \rightarrow V^*$ until convergence
- Policy function iteration (PFI): faster convergence, more complex per step

Euler-equation methods:

- Time iteration on Euler equations directly
- Endogenous grid method (EGM): invert Euler equation for speed

Aiyagari anchor: *The AI may default to grid VFI when EGM would be 50× faster on the same model. Only you know that EGM is the right call for a CRRA-utility household with a binding borrowing constraint.*

Pillar 3: Numerical Techniques

You must understand **how computation works** and where it breaks:

Core techniques:

- **Optimization:** grid search, Newton methods, choice of solver for FOCs
- **Interpolation:** linear, cubic spline, Chebyshev—accuracy vs. speed
- **Discretization:** Tauchen, Rouwenhorst for AR(1); grid design
- **Simulation:** Monte Carlo for moments, ergodic distributions

Common issues you must recognize:

- Occasionally binding constraints (borrowing limits, irreversibility)
- Curse of dimensionality
- Interpolation errors near kinks in policy functions

Aiyagari anchor: *An AI-written Aiyagari with too few grid points near the borrowing constraint produces wrong precautionary savings. The code runs, the numbers look fine, the economics is wrong.*

Pillar 4: Advanced Numerical Methods

For larger or more complex models, you need **advanced solution techniques**:

Perturbation methods:

- Linear approximation around steady state (1st order)
- Higher-order for welfare, risk premia (2nd, 3rd order)
- Fast but local—accuracy degrades far from steady state

Projection methods:

- Global approximation using basis functions (Chebyshev, finite elements)
- Handles nonlinearities and occasionally binding constraints
- Computationally intensive, requires careful convergence checks

Empirical anchor: *Smets-Wouters at the zero lower bound needs global methods (the ZLB binds and perturbation ignores it). The AI defaults to perturbation because it is fast. You must override this when the model has a kink.*

Pillar 5: Code Literacy + Division of Labor

You must **read code well enough to verify it implements your design.**

You provide (Foundations)	AI provides (Implementation)
Research question and model	Syntactically correct code
Algorithm choice and design	Translation to Python/Julia/etc.
Numerical method selection	Optimization, interpolation routines
Convergence/accuracy criteria	Diagnostic outputs and plots
Knowledge of the data and how it is set up	Data loading and cleaning
Economic interpretation of results	Debugging and refactoring

You are the architect. AI writes code; you verify it implements what you designed and validate that the answers make economic sense. Without code literacy you cannot validate; without validation, AI output is worthless.

Tool Landscape

Tool 1: Chatbox (Claude.ai) — Strengths → Workflow

Strengths:

- Large context window—can ingest papers, code, and detailed specs together
- Strong mathematical and economic reasoning
- Can discuss trade-offs (“Should I use VFI or EGM here?”)

Workflow:

1. Upload paper PDF or paste model description
2. Ask Claude to derive Bellman equation, propose algorithm, identify pitfalls
3. Iterate on the design *in chat* until you are confident
4. Hand the polished design off to Claude Code for implementation

Aiyagari anchor: *design the Bellman equation and Tauchen discretization in chat* **before** you ever open Claude Code. Chatbox is the architect’s whiteboard.

Tool 2: IDE Integration (Cursor) — Strengths → Workflow

Strengths:

- AI inside your editor, with full file context
- Tab completion as you type
- Multi-file edits via natural-language composer
- Fast feedback loop (no terminal context switch)

Workflow:

1. Open the file you are editing
2. Use Tab completion for boilerplate (loops, plotting, function signatures)
3. Use chat for “why is this loop slow?”-type questions
4. Use composer for “vectorize this VFI” multi-line edits

Empirical anchor: *filling out a regression specification with Tab completion—you type `model = sm.OLS(`, Cursor suggests the rest based on the columns it sees in your `DataFrame`.*

(Full keybindings list: see Appendix A.)

Tool 3: Autonomous Agent (Claude Code) — Strengths → Workflow

Strengths:

- Reads and writes multiple files across your project
- Runs code, observes errors, debugs autonomously
- Spawns specialized sub-agents in parallel for review
- Persists context across sessions via `CLAUDE.md`

Workflow:

1. Specify the task (English or slash command)
2. Claude plans, gets approval, then implements
3. Verify, review, fix loop runs autonomously
4. You inspect the result and provide corrections (which become memory)

Aiyagari anchor: *convert a 600-line MATLAB Aiyagari codebase to Python overnight. You wake up to working Python with a session log explaining each design choice.*

(Cross-link: alternatives in slide “Claude Code in Context.”)

The Homotopy Workflow

Start Here: The Recommended Workflow for New Users

0. Before Claude—pen + paper

Write one sentence: “I want to [compute X] using [method Y] for [question Z].” If you cannot write this, Claude cannot help yet.

1. Enter plan mode (Shift+Tab)

Describe your goal. Claude drafts a plan. *Nothing runs yet.*

2. Review and approve

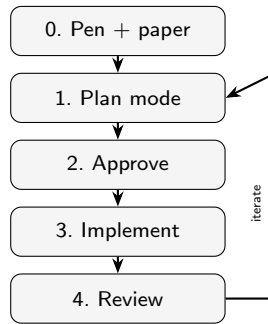
Read the plan. Ask questions. Revise. Type “approved.” *This 2-minute step saves 20 minutes of debugging.*

3. Implement

Claude executes the approved plan. Watch permission prompts. Press Esc to stop if wrong.

4. Review output

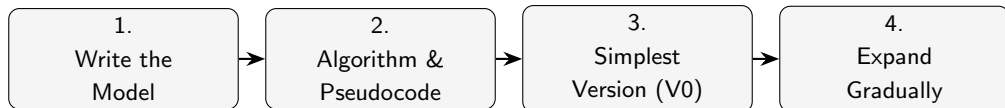
Does it make economic sense? Re-derive one



Most common beginner mistake: skipping Step 0. Without a clear goal, Claude will confidently build the wrong thing.

The Homotopy Approach to AI-Assisted Macro Research

From numerical methods: homotopy continuation starts from a problem with a known solution and continuously deforms toward the target problem. The same principle applies to AI-assisted coding.



The principle: do not ask the AI “solve my model.” Instead, *build context incrementally*—model first, algorithm second, simple version third, extensions last. Each step gives the AI more validated context to build on.

Why “homotopy”? Just as numerical homotopy traces a smooth path from a solvable problem to the target, this workflow traces a smooth path from a *specifiable* model to a *validated* codebase.

Step 1: Give the AI Enough Context

Before any code: write down your research question and model in $\text{\LaTeX}$ or Markdown.

What to write down:

- The research question in one sentence
- The full optimization problem: objective, constraints, information structure
- Equilibrium definition (if applicable)
- Parameter choices and their economic motivation
- What the output should look like (policy functions, distributions, moments)

Aiyagari anchor:

$$\max \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t u(c_t) \quad \text{s.t.} \quad c_t + a_{t+1} = (1+r)a_t + wz_t, \quad a_{t+1} \geq \underline{a}$$

with z_t following a Markov chain, $u(c) = c^{1-\sigma}/(1-\sigma)$, $\sigma = 2$, $\beta = 0.96$.

Why this matters: the quality of the code the AI writes is *directly proportional* to the quality of your specification. A vague prompt produces a model that looks right but solves the wrong problem. A 29 / 81

Step 1 in Practice: The Specification Document

Save this as `model_spec.md` or `model_spec.tex` in your project folder. The AI reads it at the start of every session.

A minimal specification for the Aiyagari model:

1. **Household problem:** CRRA utility, discount factor $\beta = 0.96$, risk aversion $\sigma = 2$
2. **Income process:** AR(1) in logs, $\rho = 0.9$, $\sigma_\varepsilon = 0.2$, discretized via Tauchen with 7 states
3. **Asset grid:** 200 points, triple-exponential spacing near $\underline{a}$
4. **Solution method:** VFI with Howard improvement (50 Howard steps every 25 VFI iterations)
5. **Convergence:** sup-norm on value function $< 10^{-8}$
6. **Equilibrium:** r clears the asset market; bisection on r
7. **Output:** policy functions $c(a, z)$, $a'(a, z)$; stationary distribution $\mu(a, z)$; Gini coefficient

Make sure the AI understood: ask it to restate the model in its own words before proceeding. Misunderstandings caught here cost minutes; misunderstandings caught in V2 cost hours.

Step 2: Algorithm and Pseudocode

Once the model is clear: work with the AI on the algorithm design. Write pseudocode together—this catches design errors *before* implementation.

Aiyagari anchor—pseudocode for V0 (deterministic):

```
Initialize: asset grid  $a[1..N]$ , guess  $V_{old} = u(r*a + w) / (1-\beta)$ 
Repeat until  $||V_{new} - V_{old}|| < tol$ :
  For each  $a_i$  in grid:
    For each  $a'$  in grid s.t.  $c = (1+r)*a_i + w - a' > 0$ :
      compute  $u(c) + \beta * \text{interp}(V_{old}, a')$ 
     $V_{new}[i] = \max \text{ over } a'$ 
     $a\_policy[i] = \text{argmax } a'$ 
  Every 25 iterations: run 50 Howard improvement steps
Return  $V_{new}$ ,  $a\_policy$ ,  $c\_policy = (1+r)*a + w - a\_policy$ 
```

What this step produces:

- Agreement on the solution method (VFI vs. EGM vs. perturbation)
- Pseudocode that serves as a *contract*—you and the AI agree on the algorithm before any Python is written
- Explicit convergence criterion and grid construction

The AI may suggest EGM is faster. Good—but *you* decide. This is Pillar 2 (Algorithm Design) in ^{31/81}

Step 3: Start Simple — The V0 Version

Implement the simplest version of the model that captures the core structure. Debug, test, validate against analytical benchmarks. **Do NOT move on until V0 passes all benchmarks.**

Aiyagari anchor: $V0$ = deterministic income ($z_t = 1$ always).

Three analytical benchmarks for V0:

1. $\beta(1+r) = 1$: constant consumption; $a'(a) = a$ (45-degree line)
2. $\beta(1+r) < 1$: agent decumulates; $a \rightarrow \underline{a}$
3. $\beta(1+r) > 1$: agent accumulates; a grows without bound

If V0 fails these benchmarks: the Bellman equation is wrong, the grid is broken, or the convergence criterion is too loose. *Everything downstream will be wrong too.*

The discipline: resist the temptation to skip ahead. A working V0 takes 30 minutes. Debugging a broken V2 because V0 was never validated takes a full day.

Step 4: Expand Gradually Until Baseline

Add one feature at a time. Each expansion must have its own benchmark.

Version	What you add	Benchmark
V0	Deterministic income	3 analytical cases above
V1	Markov income risk	Set $\sigma_z = 0$: must recover V0 exactly
V2	Stationary distribution	$\sum \mu(a, z) = 1$; mass at <u>a</u>
V3	General equilibrium	r clears asset market; Gini $\approx$ data

This IS the homotopy:

- V0 is the “easy problem” with a known solution
- Each version continuously deforms toward the research target
- Each validated version is the *foundation* for the next
- If V2 breaks, you know the bug is in V2’s new code—not in V0 or V1

When to stop: when you reach the baseline model your research question requires. The path $V0 \rightarrow V1 \rightarrow \dots \rightarrow$ baseline is your audit trail.

Why Homotopy Works with AI

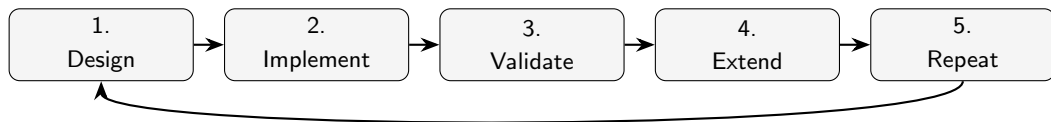
Four reasons this workflow is especially effective with AI agents:

1. **AI excels at well-specified, bounded tasks.** “Solve my model” fails. “Implement V0 per this pseudocode and validate against these 3 benchmarks” succeeds
2. **Starting simple gives both you and the AI a working codebase.** V1 inherits validated V0 code. The AI has concrete, tested functions to build on—not a blank file
3. **Bug attribution is immediate.** If V2 breaks, the bug is in V2’s new code. You and the AI can focus on the 30 new lines, not the 300 inherited ones
4. **Context builds naturally.** By V3, the AI has seen the model specification, the pseudocode, three validated versions, and all the corrections you made along the way. It has *deep context*—not because you dumped a 10-page prompt, but because you built it incrementally

The numerical-methods analogy: just as homotopy continuation avoids local optima by tracing a smooth path from a known solution, incremental model-building avoids cascading bugs by validating at every step.

The Iterative Workflow

The Five-Step Loop



Step 1: Design. Specify state space, Bellman equation, solution method, convergence criterion.
Check extensibility before any code.

Step 2: Implement. Translate algorithm to code with diagnostics (convergence plots, Euler residuals).

Step 3: Validate. Technical *and* economic checks (next slide).

Step 4: Extend. Add *one* feature; update design *before* coding.

Step 5: Repeat. Each validated version is the foundation for the next.

Pillar reference: steps 1, 4 = Pillars 1–4. Steps 2, 3 = Pillar 5.

Why This Workflow Works

1. Separating design from implementation catches conceptual errors early.

Algorithm bugs are cheaper to fix on paper than in code. Design first, implement second.

2. Clear attribution of bugs.

Adding one feature at a time means you know exactly what caused any new issue. If V1 broke after adding income risk, you know it is the new code, not the old VFI.

3. AI succeeds with structured, well-specified instructions.

Good communication makes good collaboration. Vague requests produce vague code.

4. Analytical benchmarks catch errors early.

The simplest version often has closed-form solutions. Validating against these catches fundamental errors before complexity obscures them.

Effective Prompting: Vague vs. Specific

Vague (won't work)

"Analyze this data."

"Make it faster."

"Fix the regression."

"Add some validation."

Specific (works)

"Load `employment.csv`, compute monthly growth rates by sector, plot a time series in BLS style."

"Vectorize the inner VFI loop using NumPy broadcasting; profile before/after."

"In the OLS, cluster standard errors at the firm level using `statsmodels`."

"Add an Euler-residual check; report max and mean below 10^{-6} ."

The pattern: say *exactly* which file, which variables, which package, which acceptance criterion.
Verbosity is cheap; vagueness is expensive.

Source: Paul Goldsmith-Pinkham, "Getting Started with Claude Code."

Validation: Your Core Responsibility

AI writes code. **You validate economics.** At every iteration:

Technical Checks

- Convergence achieved?
- No NaN/Inf values?
- Policy not hitting grid boundaries?
- Euler residuals small? ($< 10^{-6}$)
- Standard errors finite?

Economic Checks

- Results economically sensible?
- Comparative statics correct?
- Limiting cases recover benchmarks?
- Matches analytical solutions?
- Magnitudes match prior beliefs?

Trust but verify. AI catches indexing errors and numerical issues; *you* catch economic nonsense. AI suggests diagnostics; you run them and report back.

The boundary: if you cannot validate a result, you cannot defend it in a seminar—and the AI cannot defend it for you.

Project Structure

Naive Baseline: Chatting Without a Harness

What happens if you just open Claude Code in an empty folder and start chatting?

Four failure modes:

1. **You re-explain your project conventions every session.** “Use NumPy not lists, set seed 42, use my variable naming...”
2. **Claude forgets the correction you gave yesterday.** You corrected the asset grid bug; tomorrow it makes the same mistake.
3. **Every Aiyagari extension reverts to grid-VFI defaults**—ignoring your decision to use EGM, because the decision was never written down.
4. **No audit trail of which decisions were made why.** Six months later you cannot remember why you used Tauchen with 7 states instead of Rouwenhorst with 5.

The harness fixes all four. Files on disk are how Claude remembers, learns, and stays consistent across sessions. The next 6 slides walk through what those files are.

What a Claude Code Project Looks Like

```
1 my-aiyagari-project/  
2 +-- CLAUDE.md                # Project brain (read every session)  
3 +-- MEMORY.md                # Corrections: [LEARN:tag] entries  
4 +-- .claude/  
5 |   +-- settings.json        # Permissions for Python, git, etc.  
6 |   +-- skills/              # Workflows you invoke (/command)  
7 |   |   +-- solve-model/     # /solve-model aiyagari v0  
8 |   |   |   +-- SKILL.md     # Workflow steps  
9 |   |   |   |   +-- references/ # Algorithm specs and benchmarks  
10 |   |   |   |   |   +-- aiyagari-guide.md  
11 |   |   |   +-- validate/    # /validate aiyagari v1  
12 |   |   |       +-- SKILL.md  
13 |   +-- agents/              # Specialized reviewers  
14 |   |   +-- code-reviewer.md # Code quality, vectorization  
15 |   |   +-- domain-reviewer.md # Economic correctness  
16 |   |   +-- verifier.md      # Runs code, checks output  
17 |   +-- rules/               # Standards (always active)  
18 |       +-- plan-first-workflow.md  
19 |       +-- orchestrator-protocol.md  
20 |       +-- iterative-workflow.md  
21 |       +-- verification-protocol.md  
22 |       +-- code-conventions.md  
23 +-- scripts/                  # Python code (aiyagari_v0.py, ...)  
24 +-- output/                   # Figures, tables, results  
25 +-- quality_reports/plans/     # Plans and session logs
```

Every component you'll see for the next 10 slides has a place in this tree.

CLAUDE.md — The README for AI

CLAUDE.md is read at the start of every Claude Code session.

```
1 # CLAUDE.md --- Aiyagari Model Project
2 **Project:** AI & ML for Quant Macro
3 **Language:** Python (NumPy/SciPy)
4 **Philosophy:** Never skip validation between versions.
5
6 ## Available Skills
7 | /solve-model aiyagari | Iterative model-building workflow |
8 | /validate aiyagari v1 | Run benchmarks and convergence |
9
10 ## Available Agents
11 | code-reviewer | READ-ONLY | Vectorization, stability |
12 | domain-reviewer | READ-ONLY | Equations, equilibrium |
13 | verifier | EXECUTE | Runs scripts, checks output |
14
15 ## Model Versions
16 v0: Deterministic savings | v1: Income risk | v2: Stationary dist
```

Python analogy: CLAUDE.md $\approx$ `__init__.py` + `pyproject.toml`.

MEMORY.md — Persistent Corrections

MEMORY.md accumulates lessons learned across sessions. Each entry is a one-paragraph correction with a tag.

```
1 # MEMORY.md --- Accumulated Learnings
2
3 [LEARN:aiyagari-grid] Asset grid should use more points
4   near the borrowing constraint. Triple-exponential
5   spacing with 200 points. Linear grids need 500+.
6
7 [LEARN:vfi-howard] Howard improvement cuts VFI iterations
8   by ~80%. After every 25 VFI iterations, run 50 Howard
9   steps. Convergence tolerance: 1e-8 on sup-norm.
10
11 [LEARN:notation] Convention: V for value function,
12   c_pol for consumption policy, a_pol for savings policy,
13   Pi for transition matrix, mu for stationary distribution.
```

How an entry is created: you correct Claude (“the grid is too coarse near the constraint”). Claude saves a [LEARN:tag] entry. Next session, it reads these *before* starting work—and never repeats the mistake.

The Three Layers: Skills, Agents, Rules

	Rules	Skills	Agents
Location	<code>.claude/rules/</code>	<code>.claude/skills/</code>	<code>.claude/agents/</code>
Loaded	Auto, always	On <code>/cmd</code>	When skill/rule references
Purpose	<i>How</i> to behave	<i>What</i> to do	<i>Who</i> does sub-task
Mechanism	Claude obeys	Claude follows steps	Claude reads & adopts
Python $\approx$	Linters / config	Function	Class definition
Example	"plan first"	<code>/solve-model</code>	<code>code-reviewer.md</code>

The causal chain:

1. Rules load at session start—constrain everything
2. You invoke a skill (or give plain English)
3. The active workflow says “use the code-reviewer agent”—Claude reads the agent file and follows the checklist

Direction of control is one-way: rules $\rightarrow$ skills $\rightarrow$ agents. Agents do *not* choose skills.

English Instructions vs. Encoded SKILL.md

Confusion to clear up: What do you say in English vs. what is in SKILL.md? When does each apply?

Plain English (one-off)

"Implement the Aiyagari model with idiosyncratic income risk and a Tauchen-discretized AR(1) process."

When to use:

- First time approaching a model
- Exploring an idea
- One-off analysis you don't expect to redo

Slash command (encoded skill)

```
/solve-model aiyagari v1
```

When to use:

- You have a workflow you want to repeat exactly
- Validation steps must run identically each time
- Multiple students or co-authors must follow the same procedure

Both work; the difference is reproducibility. English is for exploration; encoded skills are for production.

Beginner Mistakes to Avoid

Mistake 1: Treating Git as Optional

Memorable line from Panjwani (2026): *“You don't need to know Git to use Git.”*

Three reasons Git matters for agentic workflows:

1. **Safe experimentation.** Git lets you try changes and roll them back. Without Git, every agent experiment is *irreversible*—one bad refactor and your working code is gone
2. **Agent memory.** Once your project has real commit history, the agent can inspect how files evolved—it can reason about when a bug was introduced by reading `git log` and `git diff`
3. **Parallel work.** Git worktrees let you work on multiple aspects of a project simultaneously—the agent can restructure code on one branch while you continue analysis on another

Practical step: put this line in your `CLAUDE.md` or `AGENTS.md`:

■Make atomic git commits as you go along.■

Source: Panjwani (2026). See also: “Why Git Makes Claude Code 10× More Powerful” (ai-mba.io).

Mistake 2: Underutilizing Skills

Skills = saved playbooks. Instead of re-explaining everything each session, you invoke the skill and the agent starts with context.

Economists have repeatable tasks:

- Review a paper (argument structure, econometric spec, citation completeness)
- Run a data-analysis workflow (load → EDA → regress → publication tables)
- Prepare a funding proposal (donor profile, context brief, draft)
- Work with Stata or Beamer in project-specific ways
- Summarize meeting notes or seminar feedback

Without skills: you re-explain the same workflow every session—the same failure mode as “chatting without a harness” (see Section 6). Claude interprets “validate” differently each time.

With skills: methodology is on disk, version-controlled, and shareable with co-authors. Your workflow becomes *reproducible infrastructure*, not ephemeral chat.

Economist-Authored Skills in the Wild

Three real examples from working economists—not toy demos:

1. **Pedro Sant'Anna's** review-paper **skill**:

- Reads the full manuscript end-to-end
- Evaluates across 6 dimensions (argument, econometrics, citations, ...)
- Generates 3–5 “referee objections” and saves to `quality_reports/`

2. **Pedro Sant'Anna's** data-analysis **skill**:

- End-to-end R workflow: `setup` → `EDA` → `main analysis` → `publication-ready tables and figures`

3. **Chris Blattman's** proposal-write **skill**:

- Drafts funding proposals from structured inputs
- Auto-gathers project context, builds donor profile, produces a polished draft

Sources: github.com/pedrohcgsc/claude-code-my-workflow;
github.com/chrisblattman/claudeblattman

Mistake 3: One Giant Thread (Context Pollution)

The symptom: “Codex got dumber” or “Claude got dumber” after 30 turns.

The real problem: *context pollution*, not model collapse. A long thread mixes planning, implementation, debugging, and revision. The context window fills with stale intermediate thoughts that are no longer relevant—but the model still pays attention to them.

The fix—a two-session workflow:

1. **Plan session:** discuss the design, iterate on the approach, reach agreement
2. **Write plan to a file:** save the agreed plan as `plan.md`
3. **Fresh execution session:** start a new conversation and execute only the plan

Why this works: the execution session’s context window is entirely available for implementation. No stale planning artifacts, no abandoned debug logs, no 15-turn-old constraint that the model has forgotten.

Cross-link: this connects directly to the `/compact` strategy in the “Context Window Mechanics” slides—both are about *protecting the context window for the work that matters now*

Skills, Agents, Sub-agents

Naive Baseline: Why Encode Workflows at All?

What if you just type the workflow steps in English every session?

Four failure modes:

1. **You forget step 4 the third time.** Last week you remembered to run Euler residuals; this week you don't.
2. **Claude interprets “validate” differently each time.** Once it means convergence checks; once it means a comparison plot; once it means a unit test. Inconsistent.
3. **No version control on your methodology.** If your workflow improves, the improvement lives in your head, not in a file you can diff.
4. **Impossible to reproduce a colleague's approach.** Your co-author has a different mental workflow—now you have two methodologies and no source of truth.

Skills fix all four—by writing the methodology to disk, once, in a place Claude reads every time.

Skills = Callable Workflows

A **skill** is a multi-step workflow invoked by name—like a Python function.

Python function:

- Defined once, called by name
- Takes arguments
- Executes a defined sequence
- Calls other functions/classes

Claude Code skill:

- Defined in SKILL.md
- Invoked: `/skillname arg`
- Follows step-by-step instructions
- References agent files by name

Aiyagari example:

```
/solve-model aiyagari v1
```

runs the same 5-step methodology every time: design → implement → verify → review → score. The methodology is in SKILL.md; you do not retype it.

Skills are the glue. Rules constrain behavior; agents define roles; *skills* sequence them into a workflow.

Skill File Structure

A skill lives in `.claude/skills/[name]/` with a required `SKILL.md`:

```
.claude/skills/solve-model/  
├ SKILL.md                (required:  workflow steps)  
└ references/             (optional:  detailed guides)  
    ├── aiyagari-guide.md (Aiyagari algorithm details)  
    └── rbc-guide.md      (RBC algorithm details)
```

SKILL.md has two parts:

1. **YAML frontmatter**: name, description, command, arguments
2. **Markdown body**: workflow steps, validation criteria, prompting patterns

Reference files contain detailed model-specific algorithms. Claude loads them *only when needed*, keeping the context window efficient.

Agents = Role Descriptions, Not a Dispatch System

An **agent file** is a markdown document describing a specialized role—its checklist, review criteria, and report format. Claude Code does **not** have a native agent dispatcher.

What actually happens:

1. A skill (or always-on rule) says: “spawn the code-reviewer agent”
2. Claude reads `.claude/agents/code-reviewer.md`
3. Claude either (a) adopts the role itself in the current conversation, or (b) launches a subagent via the Task tool—a separate Claude instance

Three Aiyagari agents:

- `code-reviewer.md`: vectorization, NumPy broadcasting, no hardcoded paths
- `domain-reviewer.md`: Euler-equation correctness, equilibrium consistency
- `verifier.md`: actually runs `aiyagari_v1.py`, checks convergence

Why separate files? A single prompt asking about all three dimensions does a mediocre job at each. Separation enables focus.

Subagents = Parallel Child Processes

When Claude launches a subagent, it spawns a **separate Claude instance** with its own context window.

- Multiple subagents can run **in parallel**, like Python's multiprocessing
- Each receives the agent file as its prompt
- Each runs to completion and returns a report
- The main conversation receives the reports and decides what to do

Aiyagari example: after writing `aiyagari_v1.py`, the orchestrator spawns:

```
code-reviewer || domain-reviewer
```

in parallel. Both finish in roughly the same wall-clock time it took one to run sequentially. **Review time is halved.**

Python analogy: `multiprocessing.Pool().map(review, [code, domain])`.

Rules = Always-On Guardrails

Rules are the only component that requires zero user action:

- Every `.md` file in `.claude/rules/` is loaded at session start
- Claude obeys them throughout the entire session—you never invoke them
- They constrain *all* work: skill-invoked, plain English, or ad-hoc editing

Aiyagari example rules:

- `plan-first-workflow.md`: “write a plan to disk before any code”
- `iterative-workflow.md`: “never extend V0 to V1 until V0’s three deterministic benchmarks pass”
- `verification-protocol.md`: “always run the verifier agent before reporting success”
- `code-conventions.md`: “`np.random.seed(42)`, no hardcoded paths”

Python analogy: rules $\approx$ `pyproject.toml` + linter configs. They constrain all code without being called explicitly.

Hooks + Memory + Settings: Three Persistence Layers

Claude Code has three different persistence mechanisms with three different lifetimes:

Layer	Lifetime	Purpose
Hooks (<code>settings.json</code>)	Per-event	Shell scripts that fire on tool calls (decorators). Enforce things mechanically: e.g., auto-format on every Python edit
CLAUDE.md	Per-session	Bootstrap context. Read at the start of every session
MEMORY.md	Cross-session	Accumulated learnings. <code>[LEARN:tag]</code> entries persist forever

The distinction matters. Hooks are for things that must *always* happen mechanically (formatting, linting). CLAUDE.md is for context that must be *loaded* every session. MEMORY.md is for corrections that must *never* be repeated.

Rule of thumb: natural-language guidance → rules and skills. Mechanical enforcement → hooks.

Tracing Aiyagari End-to-End

Aiyagari Model: Quick Recap

Households maximize $\mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t u(c_t)$ subject to

$$c_t + a_{t+1} = (1 + r) a_t + w z_t, \quad a_{t+1} \geq \underline{a}$$

where z_t is idiosyncratic productivity following a Markov chain.

Three deterministic benchmarks based on $\beta(1 + r)$:

- $\beta(1 + r) = 1$: constant consumption; $a'(a) = a$
- $\beta(1 + r) < 1$: decumulate; $a \rightarrow \underline{a}$
- $\beta(1 + r) > 1$: accumulate; a grows

Why we build it incrementally:

Version	Features	Benchmark
v0	Deterministic income	3 analytical cases
v1	Add Markov income	Limit case $\sigma_z = 0$ recovers v0
v2	Stationary distribution	Sums to 1, mass at constraint

Full DEQN treatment with aggregate uncertainty: Lec. 6.

Tracing /solve-model aiyagari (1/2): File Flow

```
1 my-aiyagari-project/
2 +-- CLAUDE.md           # 1. Read first: project context
3 +-- MEMORY.md           # 2. Read: past corrections
4 +-- .claude/
5 |   +-- rules/          # 3. ALL rule files auto-loaded
6 |   |   +-- plan-first-workflow.md
7 |   |   +-- orchestrator-protocol.md
8 |   |   +-- iterative-workflow.md
9 |   |   +-- verification-protocol.md
10 |   |   +-- code-conventions.md
11 |   +-- skills/
12 |   |   +-- solve-model/
13 |   |       +-- SKILL.md      # 4. Read: you typed /solve-model
14 |   |       +-- references/
15 |   |           +-- aiyagari-guide.md # 5. Read: SKILL.md says "see ref"
16 |   +-- agents/
17 |       +-- code-reviewer.md   # 6. Spawned later by orchestrator
18 |       +-- domain-reviewer.md # 6. Spawned later by orchestrator
19 |       +-- verifier.md        # 6. Runs scripts, checks output
20 +-- scripts/                   # 7. Code written here
21 +-- output/                    # 7. Results saved here
22 +-- quality_reports/plans/     # 8. Plans saved here
```

Numbers show the **reading order**. Steps 1–5 happen *before* any code is written.

Tracing /solve-model aiyagari (2/2): Orchestrator Loop

```
1 You type: /solve-model aiyagari v0
2
3 --- plan-first-workflow.md rule kicks in ---
4 Step 1: Write plan, save to quality_reports/plans/
5 Step 2: Ask for approval -> You: "Approved."
6
7 --- orchestrator-protocol.md rule activates ---
8 For v0, then v1, then v2:
9   Step 3: IMPLEMENT -> writes scripts/aiyagari_v0.py
10  Step 4: VERIFY     -> spawns verifier.md agent
11                      (runs script, checks convergence)
12  Step 5: REVIEW     -> spawns code-reviewer.md ||
13                      spawns domain-reviewer.md (parallel)
14  Step 6: FIX        -> applies suggestions (Critical>High>Medium)
15  Step 7: SCORE      -> Correctness >= 8? Done. Next version.
16
17 After v2: writes session log to quality_reports/
```

Every step maps to a specific file in the project folder. The loop is in `orchestrator-protocol.md`—a rule, not a command.

Inside CLAUDE.md for the Aiyagari Project

```
1 # CLAUDE.md --- Aiyagari Model Project
2
3 **Project:** AI & ML for Quant Macro
4 **Language:** Python (NumPy/SciPy)
5 **Philosophy:** Never skip validation between versions.
6
7 ## Folder structure
8 scripts/      Python code per version (aiyagari_v0.py, ...)
9 output/       Figures and tables
10 .claude/      Skills, agents, rules
11
12 ## Available skills
13 | /solve-model aiyagari [v0|v1|v2]      Iterative model build
14 | /validate aiyagari [v0|v1|v2]         Run benchmark battery
15
16 ## Available agents
17 | code-reviewer      READ-ONLY   Vectorization, NumPy broadcasting
18 | domain-reviewer    READ-ONLY   Euler equations, equilibrium
19 | verifier           EXECUTE     Runs script, checks convergence
20
21 ## Conventions
22 np.random.seed(42); no hardcoded paths; V/c_pol/a_pol/Pi/mu
```

Inside SKILL.md and aiyagari-guide.md

SKILL.md (workflow):

```
---
name: Solve Model
description: Iterative
  build (design, implement,
    validate, extend)
command: solve-model
arguments:
  - model      # aiyagari, rbc
  - version    # v0, v1, v2
---

## Steps
### 1. Design
Read references/{model}-guide.md
Save plan to quality_reports/

### 2. Implement
Write scripts/{model}_{version}.py
Include diagnostics

### 3. Validate
Run verifier; run domain-
reviewer; run code-reviewer.
ALL must pass.

### 4. Extend
Only after validation.
```

aiyagari-guide.md (reference):

```
# Aiyagari Reference

## u(c) = c^(1-sigma)/(1-sigma)
## sigma=2, beta=0.96

## V0: Deterministic
3 cases: beta(1+r) <=, >1
Benchmark: closed form

## V1: Markov income
Tauchen 7 states, rho=0.9
Benchmark: limit sigma=0
recovers V0; precautionary
savings positive

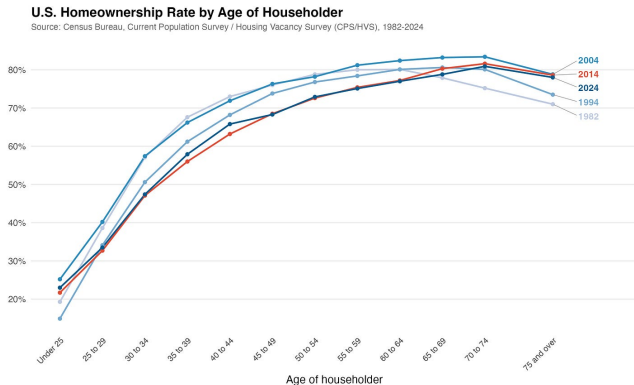
## V2: Stationary dist
Iterate forward from delta_0
Sum to 1; mass at a_min
```

Reference files load only when needed—keeps the context window efficient.

Case Study: Empty Folder → Figure

From Empty Folder to Figure

Question: “How has the age distribution of US homeowners changed over the last 50 years?” **Setup:** empty directory.



Target output (above): produced by a single Claude Code session in 6 min 21 s. *Source: Paul Goldsmith-Pinkham, “From an Empty Folder to a Figure.”*

The Exact Prompts

Prompt 1 — data discovery:

"I'm starting a new project from scratch. I want to analyze how the age distribution of homeowners in the US has changed over the last 50 years. Download homeownership rates by age going back as far as possible. Write this as `download_data.py`. Save the raw data as a CSV."

Prompt 2 — figure creation:

"Please make a script to generate a figure in R with this data. Please follow the best practices of making figures from Kieran Healy."

Prompt 3 — iteration:

"This is great, but I want a graph where age is on the x-axis and the distribution is on the y-axis and there are different lines for each year."

Three prompts, six minutes, one publication-quality figure.

Iteration Loop and Obstacles

The session was *not* a straight line. Claude hit two real obstacles and recovered from both autonomously.

Obstacle 1: FRED dead-end. Claude first searched FRED for a pre-computed age series. FRED does not have one disaggregated to brackets going back 50 years. Claude pivoted to the Census Bureau's Housing Vacancies and Homeownership Survey *without being told*.

Obstacle 2: Census 403 error. The Census website returned HTTP 403 (bot detection). Claude diagnosed it, added a browser User-Agent header to the `requests` call, and retried successfully.

Sub-agents for web research. Claude spawned helper subagents to read documentation pages, keeping the main context window clean. The subagents returned just the URLs and table numbers needed.

End-to-end timing: 6 minutes 21 seconds. Two CSVs (Census Tables 12 + 19), 77 lines of R, one figure.

What Claude Did vs. What Stayed Human

Claude did:

- Discovered Census Tables 12 and 19
- Parsed Excel by sampling structure (not row-by-row)
- Wrote `download_data.py`
- Generated 77-line R script
- Debugged Cairo font rendering
- Recovered from FRED + 403 obstacles without prompting

Human did:

- Posed the question (50 years, age distribution, US)
- Requested the axis flip (the iteration prompt)
- Judged whether the story was right
- Decided which two tables mattered

“Claude pulled the data and made the figure, but you brought the question. Research decisions remain human decisions.”

—Paul Goldsmith-Pinkham

How Research Changes + Limits

Tedious Tasks Accelerated

What gets dramatically faster:

- Data cleaning (FRED CSVs, BLS Excel files, Census Bureau tables)
- Web scraping (requests, user agents, retry logic)
- Code refactoring (notebook → modules; MATLAB → Python)
- Variable transformation (logs, growth rates, deflators)
- Format conversion (Stata .dta → pandas DataFrame)
- Boilerplate plotting (axis labels, legends, line styling)
- Debugging (read traceback, identify root cause, propose fix)

The new bottleneck: *thinking*, not *typing*.

Where you used to spend 80% of your day on data wrangling and 20% on economics, you now spend 20% on data wrangling and 80% on the question itself. **Comparative advantage shifts toward research design.**

Context Window Mechanics

What is a context window? Every message you send, Claude bundles up the entire conversation history (your prompts, its replies, every file it read, every tool output) and ships it back to itself.

Current limit (2026): approximately 200,000 tokens (~150,000 words).

The 20-turn rule. Quality degrades noticeably after 20 conversational turns. Long sessions with extensive code outputs cause the model to “lose the thread”—it forgets a constraint you mentioned 15 turns ago.

Two coping strategies:

- **Automatic compaction:** Claude summarizes history when the limit is near. Less reliable—may drop specific constraints
- **Manual compaction:** `/compact remember <focus>`—explicit, you control what survives

Source: Paul Goldsmith-Pinkham, “Getting Started with Claude Code.”

The /compact Strategy

The pattern that works: write progress to disk *before* compacting.

```
1 You: "Write a summary of what we've done today, what
2     decisions we made, and what's left to do.
3     Save it as progress.md."
4
5 Claude: [writes progress.md with bullet points]
6
7 You: /compact remember the asset grid choice and the
8     EGM decision
9
10 [Claude compacts the conversation]
11
12 [Next session]
13 You: Read progress.md and continue from where we left off.
```

Why this works: the file on disk survives the compaction (and even survives ending the session). The conversation gets shorter; the project state is preserved.

Best practice: break work into 5–10 turn sessions with clear objectives. Compact between them.

Where Claude Code Still Fails

Concrete failure modes you *will* hit on real economic-research projects:

- **Long-context drift.** On multi-hour sessions, Claude forgets a constraint mentioned early.
Mitigation: `/compact` every 20 turns
- **Silently wrong econometric edge cases.** Claude will happily run a regression with clustered SEs at the wrong level, or use a weak instrument. The output looks fine; the inference is wrong
- **Hallucinated function signatures.** Claude is trained on a snapshot of `statsmodels` from a year ago. Newer API changes (`cov_type=`, `cluster_groups=`) break in subtle ways
- **Infinite fix-loops.** On a stubbornly failing test, Claude sometimes oscillates between two “fixes,” neither of which works. Interrupt with `Esc` and re-plan
- **Happy-path code.** The script works on the first FRED download; it crashes on the second when one series has a missing observation. Tell Claude to validate edge cases explicitly

None of these are deal-breakers—they are reasons to keep humans in the validation loop.

Security: What NOT to Expose

Do not put in front of Claude Code:

- **IRB-protected human-subjects data** (interviews, consent records)
- **Personally identifiable information** (names, SSNs, addresses, dates of birth)
- **HIPAA-regulated health data** (medical records, claims data)
- **API keys, passwords, OAuth tokens** — check before each `git push`
- **Embargoed survey microdata** (NLSY, HRS restricted use, RDC datasets)

Operational rule: *“If you wouldn’t put it on Dropbox, don’t put it in front of Claude.”*

Important nuance: the *code* runs locally on your machine. The *conversation transcript* traverses the API. So the data file never leaves your laptop, but if you paste a snippet of the data into chat, that snippet does.

Health-economics, labor-economics with linked admin data: treat Claude Code the same way you treat a public Slack channel.

Working With Sensitive Data: Workarounds

"I have sensitive data" ≠ "I cannot use agents at all."

Three practical strategies (from Panjwani 2026 Q&A):

1. **Script-and-paste:** ask the agent to write profiling or analysis scripts. Run them yourself on the real data. Paste back *only the safe summary results* (means, counts, regression coefficients—never raw records)
2. **Mock data:** create anonymized or synthetic data with similar structure (same column names, same distributions, fake values). The agent develops and debugs on the mock; you apply the final script to real data
3. **Local models:** use local LLM instances or private enterprise deployments where your institution controls the data pipeline. No data leaves your machine

Coauthors who refuse Git: keep Git on *your* side anyway. Tell the agent explicitly: *"Some incoming file changes may come from collaborators outside version control."* The agent can then help you identify collaborator edits and clean up the history.

Wrap-up

Trust but Verify

A short checklist for every Claude-generated piece of empirical code:

1. **Re-derive one statistic by hand** for one row of the data. Does the number Claude reports match what you compute on paper?
2. **Cross-check against a known package.** Run the same regression with `statsmodels` and `linearmodels`. Are the standard errors identical?
3. **Sanity-check magnitudes.** Is a 0.8 elasticity plausible for this market? Does the impulse response peak at the right horizon?
4. **Read the code line by line.** If you cannot read it, you cannot defend it. Pillar 5 is non-negotiable
5. **Limit cases.** Set $\sigma_z = 0$, recover the deterministic model. Set the borrowing constraint very loose, recover the unconstrained problem

Loop back to Pillar 5. Code literacy is what makes “trust but verify” possible. Without it, you trust without verifying.

You Are the Research Architect

AI handles the bricks.
You design the structure.

The 5-pillar workflow is what makes you the architect rather than a passive consumer of code.
Without the pillars:

- You cannot specify a problem precisely → AI solves the wrong one
- You cannot evaluate an algorithm → AI picks a slow or wrong one
- You cannot diagnose a numerical bug → AI's "fixes" make things worse
- You cannot choose between methods → AI defaults to whatever it has seen most
- You cannot read the code → AI's mistakes go unchecked

Your unique value: knowing *what* to compute, *why* it matters economically, and *whether* the results are correct. AI democratizes implementation; only humans can validate.

The Compound-Learning Frame

This is not about adopting a tool. It is about building a capability.

The cost of experimentation is collapsing. That changes the return to several skills:

- **Harness engineering** (CLAUDE.md, skills, rules) → your project memory grows richer with each session
- **Prompt specificity** → each project teaches you to direct agents more precisely
- **Verification protocols** → your checklists and benchmarks accumulate across projects
- **Economic judgment** → the one input that never gets cheaper

The compounding logic: month 1 you struggle with basic prompts. Month 3 you have a skills library. Month 6 your CLAUDE.md is a full project specification. Month 12 you are directing multi-agent workflows on complex models.

The people who start earlier compound faster. That is the practical takeaway.

Key Takeaways: Four Legs of the Stool

1. **5-pillar foundations**

Theory, algorithm, numerics, advanced methods, code literacy. The throughline of this lecture

2. **Harness engineering**

CLAUDE.md + MEMORY.md + .claude/{skills, agents, rules}. The harness is what turns a chatbot into a collaborator

3. **Iterative workflow**

Design → Implement → Validate → Extend → Repeat. One feature at a time, always validate against an analytical benchmark

4. **Verification**

Trust but verify. Re-derive, cross-check, sanity-check, read line by line. Pillar 5 is non-negotiable

Master the foundations → direct AI clearly → validate rigorously.

What Changes for Economic Research

The distance from idea to result shrinks dramatically.

- Topic exploration expands—execution cost drops, so trying a new specification is cheap
- Reproducibility improves—scripts are documented automatically; the path is preserved
- Learning accelerates—lower barrier to trying a new method (DEQN, MFG, structural estimation)
- Data-to-figure cycles measured in minutes, not hours

What stays human:

- Conceptualizing research questions
- Interpreting results against economic theory
- Knowing the data and how it is set up
- Verifying statistical edge cases
- Writing the narrative

Comparative advantage shifts. Less reward for typing speed; more reward for precise problem formulation and economic judgment.

Your First Exercise: Clean Up Old Code

The best first exercise is *not* “ask an agent to do research for me.” It is: **clean up one of your old projects.**

The prompt (from Panjwani 2026—use verbatim):

```
I want your help cleaning up this project and setting up a git repo.  
First, copy it all over to a backup directory.  
Next, suggest an initial gitignore and create a first repo.  
Then, push the initial repo to a private repo on GitHub.  
Then, check out a new branch and suggest how to restructure the repo  
    to have best practices with data/code/docs/paper organization.  
Then just do it on the new branch.  
Finally, actually run any code you've modified/restructured to make  
    sure it lines up with the prior results.
```

Why each line matters:

- **Backup directory** → psychologically safe; you can always go back
- **Gitignore** → separates source from data and generated output
- **Private GitHub repo** → off-machine backup with real version history
- **New branch** → the restructuring is reversible via `git checkout main`
- **Rerun code** → forces the agent to verify the cleanup didn't break anything

How to Keep Learning

The frontier of agentic coding for economics is moving fast. Two ways to keep up:

1. Follow the economist community on X:

Alex Imas, Chris Blattman, David Yanagizawa-Drott, Ethan Mollick, Pedro Sant'Anna, Scott Cunningham, Kevin Bryan, Andy Hall, Soumitra Shukla, Alexander Kustov, Antonio Mele, Anthony Lee Zhang, Aniket Panjwani

Why X specifically? The developers pushing agentic workflows forward are on X. Economists who are early to agentic coding are downstream of those developer conversations. LinkedIn has useful discussion; *the frontier is on X*.

2. Lower-noise paths:

- Economics of AI newsletter: ai-mba.io/newsletter/econ
- AI MBA economist cohort: ai-mba.io/cohorts/claude-code-economists
- Paul Goldsmith-Pinkham's Substack: paulgp.substack.com

Looking Ahead: Lec. 10 Case Studies

Lec. 10 will walk through four full case studies:

1. **Literature review automation** — LLM + RAG to search and summarize papers
2. **Structured data extraction** — large policy datasets (FOMC minutes, Congressional records)
3. **From empty folder to figure** — end-to-end empirical workflow (revisited in detail)
4. **Computational macro model** — Aiyagari → extension via Claude Code

The lab (1 hour): students replicate one case study end-to-end with their own `.claude/` project.

Required for the lab:

- Claude Code installed (per the Setting Up section)
- Pro or Max subscription
- A skeleton `CLAUDE.md` (template in Appendix B)

Appendix

Appendix A: Cursor and Claude Code Keybindings

Cursor (IDE-integrated):

Action	Shortcut
Inline accept	Tab
Chat (file-aware)	Cmd+L
Composer (multi-file)	Cmd+I
Inline edit	Cmd+K

Claude Code (terminal):

Action	Command / key
Compact session	/compact
Clear session	/clear
Show cost	/cost
Interrupt Claude	Esc
Plan mode	Shift+Tab
Slash menu	/

(Moved here from the main deck per advisor feedback.)

Appendix B: Minimal CLAUDE.md and SKILL.md Templates

Minimal CLAUDE.md (5 lines is enough to start):

```
# CLAUDE.md
**Project:** [your project name]
**Language:** Python (NumPy, pandas)
**Convention:** np.random.seed(42); no hardcoded paths
**Goal:** [one sentence: what are you computing?]
```

Minimal .claude/skills/my-skill/SKILL.md:

```
---
name: My Skill
description: One-sentence purpose. Used for routing.
command: my-skill
---
# Steps
1. Read the input data from data/raw/
2. Run the cleaning function
3. Save to data/clean/
4. Run validate_columns() and report any failures
```

File locations: CLAUDE.md in project root | Skills in .claude/skills/<name>/SKILL.md |
Agents in .claude/agents/<name>.md | Rules in .claude/rules/<name>.md

Appendix C: Resources and Links

Articles by Paul Goldsmith-Pinkham (Yale SOM):

- “Getting Started with Claude Code: A Researcher’s Setup Guide”
`paulgp.substack.com/p/getting-started-with-claude-code`
- “From an Empty Folder to a Figure using Claude Code”
`paulgp.substack.com/p/from-an-empty-folder-to-a-figure`

Anthropic documentation:

- Claude Code docs: `docs.anthropic.com/claude-code`
- Skills reference: `docs.anthropic.com/claude-code/skills`

Aniket Panjwani (CEPR/VoxDev, March 2026):

- “AI Agents for Economics Research” (webinar + slides)
`ai-mba.io/tutorials/ai-agents-for-economics-research`

Community resources:

- Markus Karbacher, “Claude Code for Academics” (presentation + code)